Fit-Wear

E-commerce platform — technical documentation

Stack React 19 · Vite 7 · Redux Toolkit · Supabase (PostgreSQL) · Stripe · Vercel

Live e-store-snowy-rho.vercel.app

Repository github.com/teknoninja/E-STORE

Catalog 573 products · 4 categories · 14 sub-categories · 1,412 reviews

Document covers architecture, data model, authentication, payments, every source file, deployment, and interview preparation

Contents

1. **System overview**
    1. What the application does
    2. Architecture
    3. Why this architecture
2. **Technology stack**
3. **Data model**
    1. Catalog tables
    2. User tables
    3. Row Level Security
    4. Triggers
4. **Authentication — complete flow**
    1. Registration
    2. Login and session
    3. Route protection
    4. Password reset
5. **Catalog and product flow**
6. **Cart, wishlist and cross-device sync**
7. **Checkout and orders — complete flow**
8. **Search**
9. **Security model**
10. **File-by-file reference**
    1. Root and configuration
    2. Serverless functions
    3. Client entry and routing
    4. State and data layer
    5. Pages
    6. Components
    7. Styling
    8. Database SQL
    9. Tooling scripts
    10. Legacy Sanity Studio
11. **Deployment**
12. **Known state and limitations**
13. **Interview questions and answers**
    1. React and frontend
    2. State management
    3. Authentication and security

4. Database and SQL
5. API and backend
6. Payments
7. Build and deployment
8. System design and trade-offs

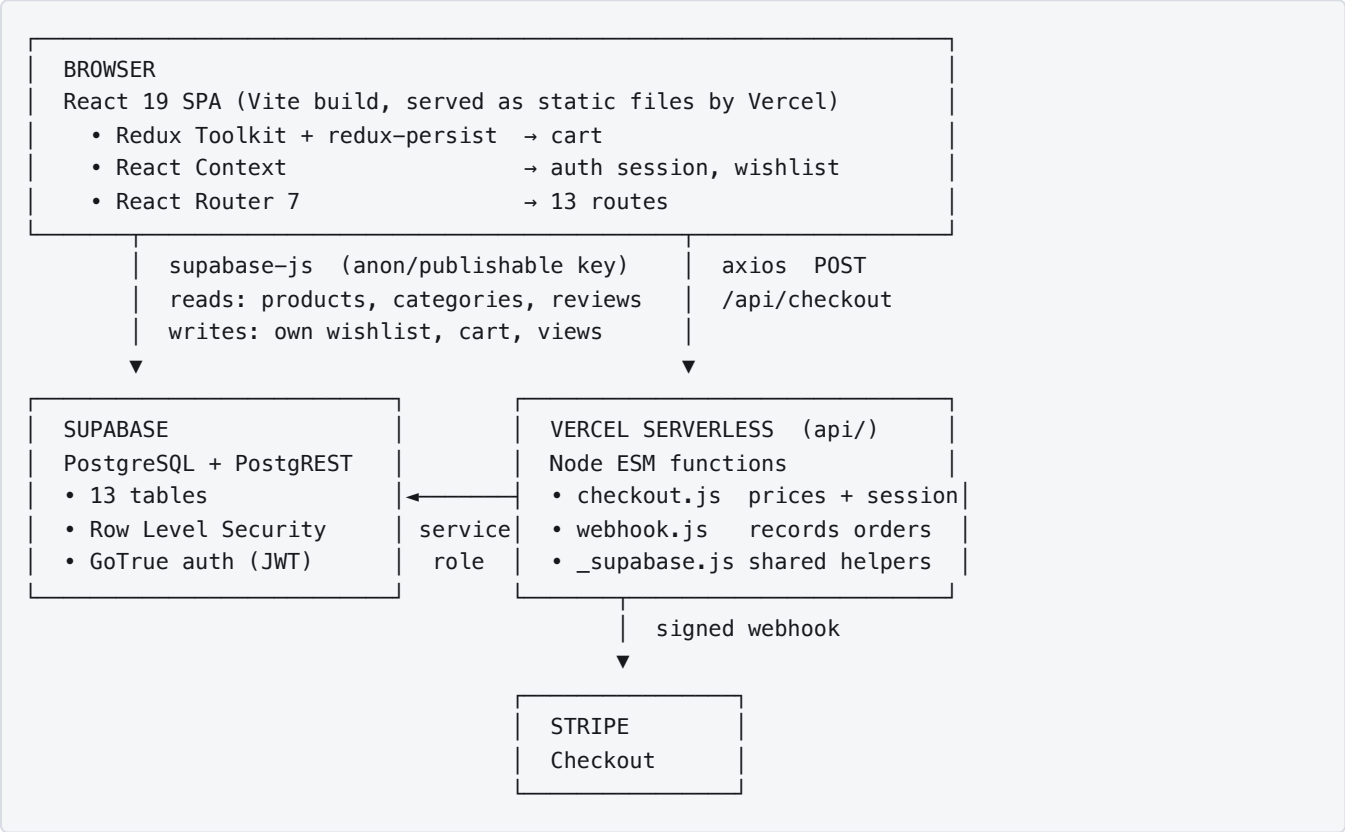
1. System overview

1.1 What the application does

Fit-Wear is a clothing and footwear storefront. A visitor can browse a catalog of 573 products, filter by category, sub-category and price, search by keyword, and view product detail pages. A registered user additionally gets a persistent wishlist, a cart that follows them across devices, an editable profile with a shipping address, and an order history populated automatically after a successful card payment.

1.2 Architecture

Three deployed pieces plus two managed services:



1.3 Why this architecture

Two design decisions define the system, and both are worth being able to defend.

The browser talks to the database directly. There is no REST API layer of our own for reading products. Supabase exposes PostgreSQL over HTTP via PostgREST, and the client queries it using the publishable key. This removes an entire tier of code, but it only works because *Row Level Security* enforces authorisation inside the database. Every query is checked by Postgres against the caller's JWT. Without RLS this design would expose every customer's data.

Serverless functions exist only where trust is required. Three operations cannot happen in the browser: computing an order total, creating a Stripe session with a secret key, and marking an order paid. Those live in `api/`, run server-side, and hold the secret keys. Everything else — browsing, search, wishlist, cart — goes straight to the database.

> **The rule to remember:** the browser may read anything public and write only its own rows. Anything involving money or another user's data happens on the server. This single principle explains every file placement in the project.

2. Technology stack

Layer	Technology	Why it was chosen
UI	React 19	Component model; concurrent rendering; the existing codebase
Build	Vite 7	Native ESM dev server; ~400 ms cold start vs. webpack's seconds
Routing	React Router 7	`createBrowserRouter` data-router API
Cart state	Redux Toolkit + redux-persist	Cart must survive a page reload for guests
Auth/session state	React Context	Read-mostly, changes rarely; Redux would be overkill
Styling	Sass (SCSS) + CSS custom properties	Nesting for component scoping; variables drive theming
Icons	MUI Icons	Icon set only — no MUI component library
Database	Supabase (PostgreSQL 15)	Free tier; RLS; PostgREST; GoTrue auth included
Auth	Supabase Auth (GoTrue)	JWT issuance, password hashing, reset emails — free to ~50k MAU
Payments	Stripe Checkout	Hosted payment page; PCI scope stays with Stripe
Hosting	Vercel	Static hosting + serverless functions in one deploy
Legacy CMS	Sanity	Original product source; now retained only for the order archive

3. Data model

Thirteen tables in the `public` schema, plus Supabase's managed `auth.users`. Live row counts as documented:

3.1 Catalog tables

Table	Rows	Purpose
`products`	573	The catalog. Text primary key (see note below)
`categories`	4	Fashion, Shoes, Men, Women
`subcategories`	14	Shirts, T-shirts, Jeans, Jackets, Suits, Dresses, shoes, sports, Saree, crop top, Bags, Watches, Sunglasses, Jewellery
`category_subcategories`	24	Many-to-many: "Jeans" belongs to both Fashion and Women
`product_categories`	964	Many-to-many product → category
`product_subcategories`	577	Many-to-many product → sub-category
`product_reviews`	1,412	Rating 1–5, comment, reviewer name; `user_id` null for seeded rows

PRODUCTS — COLUMN REFERENCE

Column	Type	Notes
`id`	text PK	Not uuid — see the note below
`slug`	text unique	URL-friendly identifier
`title`	text	
`description`	text	
`price`	numeric(10,2)	Authoritative price — server re-reads this at checkout
`old_price`	numeric(10,2)	Nullable; renders the struck-through price
`brand, sku`	text	
`rating`	numeric(3,2)	Denormalised average
`review_count`	integer	Denormalised count
`stock`	integer	Checked server-side at checkout
`tags`	text[]	GIN-indexed; colour, material, fit, brand, garment type
`images`	text[]	`[0]` card image, `[1]` hover image
`is_new`	boolean	Drives the "New Season" badge
`type`	text	`featured` | `trending` | null — homepage rows
`source`	text	`sanity` (24) | `dummyjson` (49) | `authored` (500)

> **Why `id` is `text` and not `uuid`.** Products originally lived in Sanity, whose document ids are strings, and those ids were already stored inside users' saved carts, wishlists and order line items. Migrating to `uuid` would have orphaned every saved reference. A text key let the 24 migrated products keep their original Sanity ids; new rows use readable prefixes (`dj-83` , `auth-00042`). This is a deliberate trade of key elegance for data continuity — a good example of a migration constraint driving a schema decision.

3.2 User tables

Table	Rows	Purpose
`profiles`	1	1:1 with `auth.users`. Name, phone, avatar, address (jsonb)
`orders`	1	Written *only* by the Stripe webhook
`order_items`	2	Line-item snapshot: title and price at time of purchase
`wishlist`	3	Unique on `(user_id, product_id)`
`carts`	1	One row per user; cart stored as jsonb
`product_views`	14	Behavioural log; supports signed-out visitors via `session_id`

Why `order_items` snapshots title and price instead of joining to `products`: an order is a historical record. If a product's price changes or the product is deleted, a past invoice must not change with it. For the same reason `order_items.product_id` deliberately has *no* foreign key, while `wishlist.product_id` does — a wishlist entry for a deleted product is meaningless and should cascade away.

3.3 Row Level Security

RLS is enabled on all thirteen tables. It is the sole authorisation mechanism for browser traffic.

Table	SELECT	INSERT / UPDATE / DELETE
`products, categories, subcategories, joins, reviews`	public (`using (true)`)	service role only
`profiles`	`auth.uid() = id`	own row only
`orders`	`auth.uid() = user_id`	**no policy — read-only to clients**
`order_items`	via parent order's owner	**no policy**
`wishlist`	`auth.uid() = user_id`	own rows
`carts`	`auth.uid() = user_id`	own row
`product_views`	own rows	insert where `user_id is null or auth.uid() = user_id`

Verified, not assumed. RLS was confirmed by attempting writes with the publishable key. Both were rejected:

```
POST /rest/v1/products  → 401
{"code":"42501","message":"new row violates row-level security policy for table
\"products\""}

POST /rest/v1/wishlist  → 401  (same policy violation)
```

An empty table looks identical whether RLS is on or off, so a rejected write is the only real proof.

3.4 Triggers

Two triggers on `auth.users`, both `SECURITY DEFINER`:

- `handle_new_user()` — creates the matching `profiles` row on signup, reading the display name from `raw_user_meta_data`. Guarantees the app never sees a user without a profile.

- `claim_guest_orders()` — if someone checked out as a guest and later registers with that same email, their past orders are attached to the new account.

4. Authentication — complete flow

Supabase Auth (GoTrue) handles password hashing (bcrypt), JWT issuance and email delivery. The application never sees or stores a password.

4.1 Registration

```
Register.jsx
  │  validate locally: length ≥ 8, passwords match
  ▼
AuthContext.signUp({ email, password, fullName })
  │
  ▼
supabase.auth.signUp({
    email, password,
    options: { data: { full_name }, emailRedirectTo: "/auth/callback" }
})
  │
  ▼  GoTrue: bcrypt-hash password → INSERT auth.users
  │
  ├─► TRIGGER handle_new_user()      → INSERT public.profiles
  └─► TRIGGER claim_guest_orders()   → adopt any guest orders with this email
  │
  ▼
Response contains a session?
  ├── yes → user is logged in, navigate("/")
  └── no  → "Confirm email" is enabled; show "check your inbox"
```

The `needsEmailConfirmation` flag returned by `signUp` is derived from whether a session came back, so the UI adapts automatically to the project's email-confirmation setting without a code change.

4.2 Login and session persistence

```
Login.jsx  →  AuthContext.signIn  →  supabase.auth.signInWithPassword
                                              │
                                              ▼
                          GoTrue verifies bcrypt hash
                          issues  access_token (JWT, 1 h)
                                + refresh_token
                                              │
                                              ▼
                          stored in localStorage by supabase-js
                                              │
                                              ▼
              onAuthStateChange fires → AuthContext setState → app re-renders
```

`AuthContext` subscribes once on mount and holds the single source of truth:

```
// two-step initialisation
supabase.auth.getSession()          // whatever is already in localStorage
supabase.auth.onAuthStateChange()   // keep in sync: login, logout, token refresh
```

> **Why `loading` starts `true`.** Reading the stored session is asynchronous. Without a loading flag, a logged-in user hitting refresh on `/profile` would be bounced to `/login` for one frame before their session resolved. `ProtectedRoute` renders a spinner while `loading` is true rather than deciding early.

4.3 Route protection

`ProtectedRoute` wraps three routes and does three things:

1. If `loading` — render a placeholder, decide nothing.
2. If not logged in — `<Navigate to="/login" state={{ from: pathname }} />`.
3. Otherwise render children.

The `from` in router state is what lets `Login` return the user to the page they originally wanted, rather than dumping everyone on the homepage.

> **Client-side route guarding is UX, not security.** A user can edit their JavaScript and render `/profile`. It would be empty, because the *data* is protected by RLS in the database. Never rely on a client route guard to protect information — this is a common interview probe.

4.4 Password reset

```
/forgot-password
  → supabase.auth.resetPasswordForEmail(email, { redirectTo: "/reset-password" })
  → GoTrue emails a one-time recovery link (valid 1 hour)
  → user clicks → lands on /reset-password with a recovery token in the URL
  → supabaseClient has detectSessionInUrl:true → token exchanged for a session
  → ResetPassword.jsx sees isLoggedIn → allows supabase.auth.updateUser({ password })
  → link already used / expired → no session → "Link expired" screen
```

The success message deliberately reads "if an account exists for that address" rather than confirming whether the email is registered. Confirming it would allow an attacker to enumerate the customer list.

5. Catalog and product flow

All catalog reads pass through `client/src/lib/products.js`, which is the only file that knows the database column names. It maps rows into the shape the components expect:

```
// database row          →  component shape
{ id, old_price, is_new, images }  →  { _id, oldPrice, isNew, img, img2 }
```

> **Why the mapping exists.** The components were originally written against Sanity, whose fields are `_id`, `img`, `oldPrice`. Keeping the mapper means the migration from Sanity to Postgres touched one file rather than twelve — and, critically, product ids already saved in users' carts still resolve.

THE CATEGORY LISTING QUERY

```
supabase
  .from("products")
  .select("id,title,price,old_price,is_new,images, product_categories!inner(category_id)")
  .eq("product_categories.category_id", categoryId)
  .lte("price", maxPrice)
  .order("price", { ascending: sort === "asc" })
  .range(from, to)
```

Three details matter here:

- `!inner` forces an INNER JOIN. Without it PostgREST returns *every* product with an empty join array, so the filter silently does nothing.
- Columns are listed explicitly. `select("*")` over 573 products would ship descriptions and tag arrays nobody reads.
- `range(from, from + pageSize)` over-fetches by exactly one row. If that extra row exists there is another page — one query answers both "give me a page" and "is there more", with no separate `COUNT`.

PRODUCT DETAIL PAGE

`Product.jsx` fetches one row, then fires `logProductView()` — deliberately not awaited, and its errors are swallowed. Analytics must never delay or break rendering the product.

6. Cart, wishlist and cross-device sync

Cart — a deliberately hybrid design

Concern	Where it lives	Why
Working state	Redux Toolkit	Synchronous reads; the badge must update instantly
Guest persistence	redux-persist → localStorage	Cart survives reload without an account
Cross-device	`carts` table	Cart follows the user between machines

`useCartSync` reconciles the two stores:

```
login        → fetch server cart → MERGE with local cart → dispatch setCart
cart change  → debounce 800 ms → upsert to carts table
logout       → dispatch setCart([])   (only if a cart had been hydrated)
```

The merge keeps the *larger* quantity per product rather than summing. Summing would double-count a cart the user already had open in two tabs; overwriting would silently discard one side.

> **Two ordering bugs the hook has to avoid.** (1) The push effect checks `hydratedFor.current === user.id` before writing — otherwise an empty local cart would overwrite the saved server cart on the first render after login. (2) The logout branch only clears when a cart *had* been hydrated, or every page load would wipe a guest's cart. Both are classic effect-ordering traps.

Wishlist

Server-backed via `WishlistContext`, with optimistic updates: the heart fills immediately, and the change is rolled back if the write fails. Signed-out clicks return `AUTH_REQUIRED`, which the calling component turns into a redirect to `/login` carrying the product path — so after logging in the user lands back on the product they were trying to save.

7. Checkout and orders — complete flow

This is the most security-sensitive path in the application. It is verified working in production: the live order is `paid`, $120.00, attached to a user id, with a Stripe session id and two line items.

```
1  Cart.jsx
   ├─ not signed in? → redirect to /login (an order needs an owner)
   └─ POST /api/checkout
        body:    { products: [{ _id, quantity }] }
        headers: Authorization: Bearer <supabase access_token>

2  api/checkout.js                                    [SERVER]
   ├─ getUserFromRequest(req)
   │    supabaseAdmin().auth.getUser(token)
   │    → verifies the JWT SIGNATURE. A forged token is rejected here.
   │    → null simply means guest; not an error
   ├─ SELECT id,title,price,images,stock FROM products WHERE id IN (...)
   │    ► the posted cart is treated as ids + quantities ONLY
   │    ► prices are re-read from the database
   ├─ quantity = clamp(parseInt(q), 1, 99)
   ├─ reject if stock < quantity
   └─ stripe.checkout.sessions.create({
          line_items, mode: "payment",
          metadata: { userId },          ← how the webhook knows the owner
          customer_email, shipping_address_collection,
          success_url: origin + "/orders?success=true"
      })

3  Browser → stripe.redirectToCheckout(sessionId) → Stripe-hosted page
   Card details never touch our servers. PCI scope stays with Stripe.

4  Stripe → POST /api/webhook                         [SERVER]
   ├─ read the RAW body (bodyParser disabled — see note)
   ├─ stripe.webhooks.constructEvent(raw, signature, STRIPE_WEBHOOK_SECRET)
   │    ► proves the request genuinely came from Stripe
   ├─ listLineItems(session.id, { expand: ["data.price.product"] })
   ├─ UPSERT orders  (onConflict: stripe_session_id)  status = 'paid'
   ├─ DELETE + INSERT order_items
   ├─ clear the user's saved cart
   └─ best-effort copy into Sanity (failure logged, never fatal)
```

THREE DEFENCES WORTH EXPLAINING IN AN INTERVIEW

1. **Prices are never trusted from the client.** The original implementation used `item.price` from the request body — a user could edit it and buy anything for $0.01. The server now treats the cart as a list of ids and quantities and re-reads price from the database.
2. **Identity comes from a signed token, not the body.** The client sends its access token; the server verifies the signature with Supabase before trusting the user id. A client-supplied `userId` field would be trivially forged.
3. **Orders are write-protected at the database level.** There is no INSERT or UPDATE policy on `orders`. Only the webhook, using the service-role key that bypasses RLS, can create one — and only after verifying Stripe's signature. A user cannot mark their own order paid.

> **The `bodyParser: false` bug.** Stripe signs the *raw* request bytes. Vercel parses JSON bodies by default, and a re-serialised body produces different bytes — so every signature check failed and no order was ever saved. The fix is `export const config = { api: { bodyParser: false } }` in `api/webhook.js`. This is a genuinely common production bug and a good story to be able to tell.

Idempotency. Stripe retries webhooks. `orders` upserts on the unique `stripe_session_id`, and `order_items` are deleted before insert — so a retry updates rather than duplicating. The Supabase write returns HTTP 500 on failure *so that* Stripe retries; the Sanity copy only logs, so a Sanity outage cannot cause an endless retry loop.

8. Search

Keyword search across title, brand, description and tags, with live suggestions in the navbar (250 ms debounce) and a full `/search?q=` results page.

```
const clauses = [
  `title.ilike.*${q}*`,
  `brand.ilike.*${q}*`,
  `description.ilike.*${q}*`,
];
if (!q.includes(" ")) clauses.push(`tags.cs.{"${q}"}`);   // note the quotes

supabase.from("products").select(CARD_COLUMNS).or(clauses.join(",")).limit(48)
```

THREE IMPLEMENTATION NOTES

- **ILIKE rather than full-text search.** Postgres FTS matches whole tokens, so "sneak" would return nothing. Shoppers type partial words. At 573 rows the sequential scan is imperceptible; a `pg_trgm` index is the upgrade path at tens of thousands of rows.
- **Input is sanitised.** PostgREST's `or=` is a comma-separated expression, so unescaped commas or parentheses would let a user inject additional filters. `sanitizeQuery` strips `, ( ) * \ % : . " '`.
- **The array-contains clause needs quoting.** `tags.cs.{shirt}` inside `or()` raises *"malformed array literal"*; it must be `tags.cs.{"shirt"}`. Without this every search returned a 400.

Results are ranked client-side: exact title match, then title prefix, then title substring, then everything else.

9. Security model

Three classes of key

Key	Where it lives	Exposure
`VITE_SUPABASE_ANON_KEY`	`client/.env`	PUBLIC compiled into the JS bundle by design
`VITE_STRIPE_PUBLISHABLE_KEY`	`client/.env`	PUBLIC intended for browsers
`SUPABASE_SERVICE_ROLE_KEY`	root `.env` / Vercel	SECRET bypasses ALL RLS
`STRIPE_SECRET_KEY`	root `.env` / Vercel	SECRET can move money
`STRIPE_WEBHOOK_SECRET`	root `.env` / Vercel	SECRET authenticates Stripe

> **The `VITE_` prefix is a security boundary, not a naming convention.** Vite inlines *only* prefixed variables into the bundle. This is exactly why the two `.env` files are separate directories: `client/.env` is inside Vite's root and is read at build time; the project-root `.env` is invisible to the bundler and is read only by Node scripts and serverless functions. One misplaced prefix would publish the service-role key to every visitor.

Defence summary

Threat	Mitigation
Reading another user's orders or address	RLS `auth.uid() = user_id`, enforced by Postgres
Faking a paid order	No client write policy on `orders`; webhook + signature only
Buying a $200 jacket for $0.01	Server re-reads price from the database
Forging identity at checkout	JWT signature verified server-side
Forged webhook calls	`stripe.webhooks.constructEvent` HMAC check
Filter injection via search	Input sanitisation before building the `or=` expression
Card data breach	Cards never reach our servers — Stripe-hosted page
Customer email enumeration	Password reset never confirms whether an account exists

10. File-by-file reference

10.1 Root and configuration

File	Lines	Purpose
`package.json`	14	Dependencies for `api/` (stripe, supabase-js, sanity). `"type":"module"` — required, because `api/*.js` use ESM syntax which Node rejects in `.js` without it
`vercel.json`	19	Build config: installs root + client, builds client, output `client/dist`; SPA rewrite excluding `/api`; function memory/timeout. Schema-validated — Vercel rejects unknown keys, including `"//"` comment keys
`.gitignore`	—	Excludes both `.env` files (pattern `.env` matches at any depth), `node_modules`, `dist`
`.env`	—	SECRET Server keys. Gitignored. Read by scripts and functions
`client/.env`	—	PUBLIC Vite build-time variables
`client/vite.config.js`	19	React plugin; dev proxy for `/api`
`client/index.html`	13	SPA shell with `#root`
`client/eslint.config.js`	29	Flat config: React Hooks + Refresh rules

10.2 Serverless functions — `api/`

File	Lines	Purpose
`_supabase.js`	61	Shared helpers. Leading underscore keeps Vercel from exposing it as an endpoint. `supabaseAdmin()` builds a service-role client and normalises the URL (trims whitespace/quotes, adds a missing scheme, strips a trailing `/rest/v1`). `getUserFromRequest()` verifies a Bearer JWT and returns null rather than throwing
`checkout.js`	91	Verifies the caller, re-prices the cart from the database, clamps quantities, checks stock, creates the Stripe session with `metadata.userId`
`webhook.js`	168	Verifies Stripe's signature against the raw body, writes the order and items to Supabase (idempotent), clears the saved cart, best-effort copy to Sanity. Sanity client is built lazily so missing Sanity config cannot crash the function on import

10.3 Client entry and routing

File	Lines	Purpose
`src/main.jsx`	17	Mounts React; wraps in Redux `Provider` and `PersistGate`
`src/App.jsx`	93	Provider nesting (`AuthProvider` → `WishlistProvider` — wishlist loads per user) and all 13 routes. The router is defined at module scope: recreating it each render would remount every page and lose state

ROUTE TABLE

Path	Component	Access
`/`	Home	public
`/products/:id`	Products	public — category listing
`/product/:id`	Product	public — detail
`/search`	Search	public
`/login`	Login	public
`/register`	Register	public
`/forgot-password`	ForgotPassword	public
`/reset-password`	ResetPassword	public — needs a recovery token
`/auth/callback`	AuthCallback	public — email-confirmation landing
`/profile`	Profile	**protected**
`/orders`	Profile (orders tab)	**protected**
`/wishlist`	Profile (wishlist tab)	**protected**

10.4 State and data layer

File	Lines	Purpose
`context/AuthContext.jsx`	154	Single source of truth for the session. Exposes `user, profile, loading, isLoggedIn, signUp, signIn, signOut, sendPasswordReset, updatePassword, updateProfile`. Every action returns `{ error }` so forms show messages inline
`context/WishlistContext.jsx`	113	Server-backed wishlist with optimistic add/remove and rollback. Returns `AUTH_REQUIRED` for guests
`redux/store.js`	36	Redux store; `redux-persist` with serialisable-check exemptions for its internal actions
`redux/cartReducer.js`	40	`addToCart, removeItem, resetCart, setCart`. `setCart` exists for the merge-on-login path
`hooks/useCartSync.js`	110	Merge on login, debounced push on change, clear on logout
`lib/supabaseClient.js`	30	Configured client (`persistSession` , `autoRefreshToken` , `detectSessionInUrl` , PKCE). Throws at startup if env vars are missing — a loud failure beats a confusing fetch error
`lib/products.js`	258	Every catalog read: categories, sub-categories, listings with filters and paging, single product, reviews, search, view logging. Owns the row→component mapping
`hooks/useFetch.js`	32	LEGACY unused axios hook from the pre-Supabase version
`sanityClient.js`	8	LEGACY unused since the catalog migration
`lib/recommendations.js`	38	UNUSED wrappers for recommendation RPCs; feature not enabled

10.5 Pages

File	Lines	Purpose
`Home/Home.jsx`	19	Slider, featured row, categories, trending row, contact
`Products/Products.jsx`	127	Category listing shell: sub-category checkboxes, price slider, sort radios. Resets filters when the category changes, so a stale sub-category cannot silently empty the list
`Product/Product.jsx`	129	Detail page: image switcher, quantity, add-to-cart, wishlist toggle, fire-and-forget view logging
`Search/Search.jsx`	118	Results page driven by `?q=` so results are shareable and the back button works. Suggestion chips when empty
`Login/Login.jsx`	95	Email + password, show/hide toggle, inline errors, redirect-back after login
`Register/Register.jsx`	141	Local validation before the network call; handles both email-confirmation modes
`ForgotPassword/…`	74	Sends the reset link; message avoids confirming account existence
`ResetPassword/…`	133	Consumes the recovery session; distinguishes "expired link" from "not signed in"
`AuthCallback/…`	28	Waits for `detectSessionInUrl` to settle, then forwards
`Profile/Profile.jsx`	335	Three tabs — details form (name/phone/address), order history with line items, wishlist grid with add-to-cart and remove. `defaultTab` lets `/orders` and `/wishlist` reuse the page

10.6 Components

File	Lines	Purpose
`Navebar/Navbar.jsx`	136	Sticky bar: category links from the database, static links, stores dropdown, search, user menu, wishlist count, cart drawer with outside-click close
`SearchBar/SearchBar.jsx`	163	Icon expands to an input; debounced live suggestions; Enter → `/search`. The expanded form is absolutely positioned so it cannot reflow the navbar
`UserMenu/UserMenu.jsx`	93	Icon when signed out, avatar + dropdown when signed in; closes on outside click or Escape
`Card/Card.jsx`	73	Product card with hover image swap and wishlist heart. The heart calls `preventDefault` / `stopPropagation` because the whole card is a link
`List/List.jsx`	78	Filtered grid with "load more". Depends on a joined string of `subCats`, not the array — a new array identity each render would loop the effect
`Cart/Cart.jsx`	101	Drawer: items, remove, subtotal, checkout (sends the access token), reset
`ProtectedRoute/…`	30	Auth gate with a loading state and redirect-back
`FeaturedProducts/…`	53	Homepage row filtered by `type`
`Categories/…`	37	Category tiles from the database
`Slider/Slider.jsx`	60	Auto-advancing hero carousel
`Footer, Contact`	57 / 29	Static content
`GoogleButton, ProductRail`	33 / 78	UNUSED OAuth removed; rails not enabled

10.7 Styling

SCSS per component plus a token layer. `index.css` defines the dark theme as CSS custom properties (`--bg`, `--surface`, `--text`, `--accent`, …) that every stylesheet references, so a single value change restyles the site. It also sets `color-scheme: dark` — without it, scrollbars and form controls render in light

mode against a black page — and `accent-color` so checkboxes, radios and the price slider pick up the brand blue.

Notable: `Card.scss` swaps hover images with `opacity` rather than `z-index`, and gives each image an opaque background. Imported product photos are transparent WebP; `z-index` only reorders stacking, so the image underneath showed through the cut-out and both products appeared at once.

10.8 Database SQL — `supabase/`

File	Lines	Purpose
`schema.sql`	211	User side: profiles, orders, order_items, wishlist, carts; RLS policies; both signup triggers. Idempotent
`products-schema.sql`	279	Catalog: products, categories, sub-categories, join tables, reviews, views; indexes (GIN on tags, full-text, price, type); RLS; the wishlist foreign key; commented pgvector block
`recommendations.sql`	260	NOT DEPLOYED Five recommendation functions (content-based, collaborative, personalised, recently-viewed, trending) with cold-start fallbacks

10.9 Tooling scripts

File	Lines	Purpose
`scripts/generate-catalog.mjs`	366	Authors 500 products and fetches one unique photo each from Unsplash (one search per garment type). Seeded RNG so re-runs are identical. Deletes the old duplicate rows, inserts products + joins + reviews
`scripts/catalog-taxonomy.mjs`	238	The vocabulary: garments, materials, colours, brands, description voices per garment kind. Materials and modifiers are *scoped per garment* — an unconstrained first pass produced "Corduroy Running Shoes"
`scripts/import-products.mjs`	623	The earlier migration: pulled Sanity products into Postgres and imported the DummyJSON apparel set
`scripts/push-env-to-vercel.sh`	75	Pipes local env values into `vercel env add` so secrets never enter shell history

10.10 Legacy Sanity Studio — `studio/`

The original CMS. Still holds the 24 source products and an order archive, but the site no longer reads it — edits in Studio do not appear on the storefront. Retained because the Stripe webhook still writes a best-effort order copy there for the Studio order view.

11. Deployment

One Vercel project serves both the static bundle and the functions. Root Directory must be **blank**: setting it to `client` builds the frontend but leaves `api/` outside the deployment, so every checkout would 404.

```
installCommand:   npm install && npm install --prefix client
buildCommand:     npm run build --prefix client
outputDirectory:  client/dist
rewrites:         /((?!api/).*)  →  /index.html
```

The rewrite is the SPA fallback — without it, refreshing `/profile` or opening a `/product/…` link directly returns 404, because no such file exists on disk. The negative lookahead stops API calls being swallowed by `index.html`.

Variable	Scope	Consumer
VITE_SUPABASE_URL	build	browser — app cannot boot without it
VITE_SUPABASE_ANON_KEY	build	browser — app cannot boot without it
VITE_STRIPE_PUBLISHABLE_KEY	build	browser — redirect to Checkout
SUPABASE_URL	runtime	`api/checkout.js`, `api/webhook.js`
SUPABASE_SERVICE_ROLE_KEY	runtime	both functions
STRIPE_SECRET_KEY	runtime	both functions
STRIPE_WEBHOOK_SECRET	runtime	`api/webhook.js`

> **`VITE_` variables are read at build time.** They are compiled into the bundle, so changing one requires a redeploy, not a restart — and they must exist *before* the build runs. A push that deploys without them yields a blank site, because `supabaseClient.js` throws at import and React never mounts.

Also required outside Vercel: a Stripe webhook endpoint at `/api/webhook` subscribed to `checkout.session.completed`, and the production URL added to Supabase → Authentication → URL Configuration, or password-reset links point at localhost.

12. Known state and limitations

> **Verified working in production:** registration and login, session persistence, protected routes, wishlist, cross-device cart, catalog browsing with filters and paging, keyword search, and the full payment chain — the live order is `paid`, $120.00, owned by a user, with a Stripe session id and two line items, which proves checkout, the webhook signature check, and the service-role write all work.

Limitation	Detail
Product colour vs. photo	Authored titles name a colour chosen independently of the image, so some disagree ("Black … Shirt" over a blue shirt). Fix identified: read the colour from Unsplash's `alt_description`
Checkout requires an account	Guest checkout was removed so orders always have an owner. The schema still supports guest orders and auto-claims them on signup
Sanity divergence	Studio still shows the original 24 products; edits there no longer affect the site. The 19 pre-migration Sanity orders do not appear in `/orders`
Search scope	Matches only columns on the product row — searching "shoes" does not match products linked to the Shoes *category*
No local API	`api/` functions do not run under `npm run dev`; the Vite proxy targets a port nothing listens on. Checkout is testable only on a deploy or via `vercel dev`
Denormalised ratings	`products.rating` and `review_count` are static, not recomputed from `product_reviews`
Stock not decremented	Checked at checkout but never reduced after purchase
Dead files	`GoogleButton`, `ProductRail`, `recommendations.js`, `sanityClient.js`, `useFetch.js` are imported by nothing
Bundle size	~666 kB uncompressed / ~207 kB gzipped, single chunk. Route-level `React.lazy` is the obvious improvement

13. Interview questions and answers

Questions an interviewer could reasonably ask from this project, with the answers this codebase supports.

13.1 React and frontend

1. Why is the router defined outside the `App` component?
→ `createBrowserRouter` builds a router instance. Calling it inside the component body would create a new instance on every render, remounting every route and discarding page state. Defining it at module scope creates it once.

2. `useCartSync` depends on `subCatsKey` (a joined string) rather than the `subCats` array. Why?
→ Arrays are compared by reference in a dependency list. A parent that recreates the array each render produces a new identity every time, so the effect re-runs, sets state, triggers a render, and loops. Depending on a primitive derived from the array breaks the cycle. The same class of bug appears with object and function dependencies.

3. How is the hover image swap implemented, and why not `z-index`?
→ Both images are stacked absolutely; the top one fades to `opacity: 0` on hover. `z-index` alone was the original approach and it broke: the imported photos are transparent WebP, and reordering the stack does not hide a transparent image — the one underneath showed through the cut-out. Each image also needs its own opaque background.

4. Why does every data-fetching effect use an `active` flag?
→ To avoid setting state after unmount and to avoid a race where a slow earlier request resolves after a faster later one and overwrites correct data. The cleanup sets `active = false`, and the resolver checks it before calling `setState`. `AbortController` is the alternative.

5. Why is the expanded search input absolutely positioned?
→ Rendering a 280 px input inline in a flex navbar reflowed the layout and pushed the centre title into the nav links. Taking it out of flow keeps the collapsed icon's footprint and overlays the input. Verified by measuring that the centre element's x-coordinate does not move when the input opens.

13.2 State management

6. Why Redux for the cart but Context for auth?
→ Different access patterns. The cart is written frequently from many components and benefits from Redux DevTools, reducers and the `redux-persist` integration. The session is read almost everywhere but written rarely — Context is sufficient, and a Redux slice would add boilerplate for no benefit. Worth noting the trade-off: any Context change re-renders all consumers, which is acceptable for a value that changes on login/logout.

7. Two carts exist — localStorage and a database table. How are conflicts resolved?
→ On login the two are merged, keeping the larger quantity per product. Summing would double-count a cart already open in two tabs; last-write-wins would silently discard one side. Taking the maximum loses the

least information. This is a deliberate conflict-resolution policy, and the honest answer is that no policy is perfect without per-item timestamps or CRDTs.

8. Why does the cart push to the server on a debounce?
→ Quantity buttons fire rapidly. Without debouncing, every tap is a network write. An 800 ms trailing debounce collapses a burst into one request. The push is also gated on the merge having completed, or an empty local cart would overwrite the saved one.

9. What is optimistic UI, and where is it used?
→ Applying a change locally before the server confirms it, then rolling back on failure. The wishlist does this: the heart fills instantly, a temporary row is inserted, and it is replaced with the real row on success or removed on error. It trades a small chance of a visible rollback for a large gain in perceived speed.

13.3 Authentication and security

10. The anon key is in your JavaScript bundle. Isn't that a vulnerability?
→ No — it is public by design. It identifies the project, not the user. Authorisation comes from Row Level Security: Postgres evaluates `auth.uid()` from the caller's JWT against each row. The real vulnerability would be shipping the *service-role* key, which bypasses RLS entirely.

11. What is Row Level Security and how would you prove it is switched on?
→ A Postgres feature attaching predicates to a table so every query is filtered by the current role and claims. Proof requires attempting an operation that should fail: an unauthenticated insert returns `42501 — new row violates row-level security policy`. An empty `SELECT` proves nothing, because a table with RLS disabled and no rows looks identical.

12. A user POSTs to your checkout endpoint with `userId` set to someone else's id. What happens?
→ Nothing — the endpoint ignores any user id in the body. It reads the `Authorization: Bearer` token and calls `supabase.auth.getUser(token)`, which verifies the JWT signature server-side. Identity must always be derived from a signed credential, never from request data.

13. How do you stop a user manipulating prices?
→ The server treats the posted cart as ids and quantities only, and re-reads price from the database before building the Stripe session. The original implementation trusted `item.price` from the request body, which allowed buying anything for $0.01. Quantities are also clamped to 1–99 and checked against stock.

14. Why can't a user insert their own row into `orders`?
→ There is deliberately no INSERT or UPDATE policy on that table. RLS denies by default, so the absence of a policy is the protection. Only the webhook — using the service-role key, and only after verifying Stripe's HMAC signature — can create an order.

15. What does `bodyParser: false` do and why is it required?
→ It stops the platform parsing the request body so the raw bytes remain available. Stripe's signature is an HMAC over those exact bytes; a parsed-and-re-serialised body produces a different string and every verification fails. This was a live bug in the project — signature checks failed silently and no order was ever recorded.

16. Why does the password-reset page not say whether the email exists?
→ To prevent account enumeration. A distinguishable response lets an attacker test addresses and build a customer list — useful for phishing. The generic message costs a little clarity for a real security gain.

17. What is PKCE and why use it?
→ Proof Key for Code Exchange. The client generates a random verifier, sends its hash with the authorisation request, and presents the original verifier when exchanging the code. An attacker who intercepts the code cannot use it without the verifier. It is the recommended flow for public clients, which cannot keep a secret.

13.4 Database and SQL

18. Why is `products.id` `text` rather than `uuid`?
→ A migration constraint. Product ids from the previous CMS were already stored in users' carts, wishlists and order items. Switching to `uuid` would have orphaned every saved reference. Text keys let migrated products keep their original ids. The cost is slightly larger keys and no built-in uniqueness generation — accepted for data continuity.

19. Why does `order_items` duplicate title and price instead of joining to `products`?
→ An order is an immutable historical record. Joining would make a past invoice change when a price changes, and break entirely if the product were deleted. For the same reason that column has no foreign key, whereas `wishlist.product_id` does — a wishlist entry for a deleted product should cascade away.

20. Explain the many-to-many design between products and categories.
→ A join table `product_categories(product_id, category_id)` with a composite primary key. A product belongs to several categories and a category holds many products, so neither side can carry a foreign key. `category_subcategories` is a second join table because a sub-category like "Jeans" legitimately appears under both Fashion and Women — a `parent_id` column could not express that.

21. What does `!inner` do in the PostgREST query, and what happens without it?
→ It converts the embedded resource into an INNER JOIN so filters on the joined table restrict the parent rows. Without it PostgREST performs a left join and returns every product with an empty array for non-matching joins — the category filter silently returns the whole catalog.

22. How is pagination implemented and why that approach?
→ `range(from, from + pageSize)` — offset pagination that deliberately over-fetches one row. If the extra row exists there is another page, so a single query answers both questions without a separate `COUNT(*)`. Offset pagination degrades on very large offsets; keyset (cursor) pagination on an indexed column is the scalable alternative.

23. Why ILIKE for search instead of Postgres full-text search?
→ Full-text search matches whole lexemes, so a partial word like "sneak" returns nothing, and users type partial words. ILIKE with leading wildcards cannot use a B-tree index and does a sequential scan, which at 573 rows is imperceptible. At scale the answer is a `pg_trgm` GIN index, or a generated `tsvector` column with prefix matching.

24. What index types are used and why?
→ B-tree on `price`, `type`, `brand`, `source` for equality and range filters and for `ORDER BY price`. GIN on `tags` because it is an array and the queries use containment/overlap operators, which B-tree cannot serve. GIN on a `to_tsvector` expression for future full-text search. Composite indexes on the join tables' foreign keys.

25. What is `SECURITY DEFINER` and when is it justified?
→ It runs a function with the privileges of its owner rather than the caller, allowing it to bypass RLS. Justified when a function must aggregate across users but return only non-identifying data — a "customers also bought" count, for example. It must always set an explicit `search_path`, or a caller could shadow a function name and hijack the elevated context.

26. Why put the signup logic in a database trigger rather than the application?
→ Atomicity and completeness. The profile row is created in the same transaction as the user, so the application can never observe a user without a profile — and it holds for users created outside the app, such as from the dashboard. The trade-off is that logic in triggers is less visible to developers reading only application code.

13.5 API and backend

27. Why do only three files live in `api/`?
→ Because only three operations require trust: verifying who is calling, computing an authoritative total, and marking an order paid. Everything else can go directly from browser to database, since RLS enforces authorisation there. Adding a server layer for reads would duplicate authorisation logic in two places, which is how the two drift apart.

28. Why does the filename start with an underscore?
→ Vercel maps files in `api/` to HTTP routes. A leading underscore marks it as a helper module rather than an endpoint, so `_supabase.js` is importable but not publicly callable.

29. The webhook returns 500 when the Supabase write fails but only logs when the Sanity write fails. Why the difference?
→ Stripe retries on a non-2xx response. Supabase holds the customer-facing order history, so a failure there should trigger a retry and eventually succeed. Sanity is a secondary archive; returning 500 for it would cause endless retries and duplicate Supabase rows for an outage that does not affect customers.

30. How is webhook idempotency achieved?
→ Stripe may deliver the same event more than once. The order is upserted on the unique `stripe_session_id`, and line items are deleted before insertion, so a repeat delivery updates rather than duplicating. Any webhook handler must be idempotent — at-least-once delivery is the norm.

31. Why is the Sanity client constructed inside the handler instead of at module scope?
→ Because `createClient` throws when `projectId` is missing, and at module scope that throw happens at import time and takes down the whole function. A missing optional variable would then stop orders reaching Supabase — the part that actually matters. Lazy construction keeps an optional dependency optional.

13.6 Payments

32. Why redirect to Stripe rather than collect card details yourself?
→ PCI DSS scope. Card data never touches our servers, so the compliance burden stays with Stripe. It also delivers 3-D Secure, wallets, and localisation for free. The cost is losing control of that part of the UI.

33. Why is the order created by a webhook rather than on the success redirect?
→ The redirect is not trustworthy or guaranteed. A user can close the tab, lose connectivity, or fabricate a request to the success URL. The webhook is a server-to-server call authenticated by an HMAC signature — the only reliable signal that payment actually completed.

34. How does the webhook know which user the order belongs to?
→ `checkout.js` puts the verified user id into `metadata.userId` on the Stripe session, and the webhook reads it back off the session object. The id was established from a signed JWT, so it is trustworthy even though it made a round trip through Stripe.

13.7 Build and deployment

35. What does the `VITE_` prefix actually do?
→ Vite only inlines environment variables with that prefix into the client bundle; everything else stays server-side. It is a security boundary. This is why the project keeps two `.env` files in different directories — the root one is outside Vite's root and is never read by the bundler, so the service-role key cannot be published by a stray prefix.

36. Why does the SPA need a rewrite rule, and why the negative lookahead?
→ Client-side routes have no corresponding files on disk, so requesting `/profile` directly would 404. The rewrite serves `index.html` for unknown paths and lets React Router resolve them. The `(?!api/)` lookahead excludes API paths, which must reach the functions rather than being served HTML.

37. Root Directory is blank rather than `client`. Why does that matter?
→ Root Directory defines the deployment root. Setting it to `client` would place `api/` outside the deployment, so no functions would be created and every `/api` call would 404. Instead the root stays the repository and `vercel.json` points the build at `client/`.

38. Why is `"type": "module"` required in the root package.json?
→ The files in `api/` use `import` / `export`. Node treats `.js` as CommonJS unless the nearest package.json declares `"type": "module"`, so without it the functions fail at import with *"Cannot use import statement outside a module"*.

13.8 System design and trade-offs

39. What breaks first at 100× traffic, and what would you change?
→ Search, because ILIKE with leading wildcards forces a sequential scan — a `pg_trgm` index or a real full-text column fixes it. Then offset pagination on deep pages, which keyset pagination fixes. Then the 573-row assumption behind unbounded queries. Frontend-wise the 666 kB single bundle wants route-level code splitting. The direct browser-to-database pattern actually scales well, since PostgREST is stateless and Supabase pools connections.

40. What would you do differently starting over?
→ Choose one data store on day one — the project carried a Sanity-to-Postgres migration that forced text primary keys and a field-mapping layer that still exists. Set up a local runner for the serverless functions immediately, since checkout could not be tested locally at all. And write the RLS policies before any client code, so authorisation is the foundation rather than something verified afterwards.

41. How would you add product inventory decrementing safely?
→ In the webhook, inside a transaction, using `UPDATE products SET stock = stock - $1 WHERE id = $2 AND stock >= $1` and checking the affected row count — an atomic conditional update avoids the read-modify-write race two simultaneous buyers would otherwise cause. A read-then-write in application code is the classic wrong answer. Overselling also needs a reservation at session creation, not just a decrement at payment.

42. The ratings are denormalised. What are the risks and alternatives?
→ The risk is drift: `products.rating` can disagree with the rows in `product_reviews`, and it is currently static. Options are a trigger recomputing the average on review insert or delete, a periodically refreshed materialised view, or computing it on read with a join. The trade-off is write cost and consistency against read speed — denormalisation is correct only if something keeps it true.

43. How would you build recommendations on this schema?
→ The data is already in place. Collaborative filtering by co-purchase is a self-join on `order_items` grouped by co-occurrence. Content-based similarity uses array overlap on `tags` plus shared category and price proximity. Personalisation joins `product_views` and `wishlist` into a weighted tag profile. Because products and behaviour share one database, each is a single SQL query rather than a cross-service join. Semantic similarity would add `pgvector` with embeddings. The important design point is the cold-start problem: collaborative filtering returns nothing until orders exist, so every query needs a fallback.